

Summary of Inference: Disaggregation and Fusion (September 30 2026)

Max Tang mhtang@umich.edu Jagger Pacheco pacheco@umich.edu

Lars Wachter wlars@umich.edu Srinithish Durai sridurai@umich.edu

1 Research direction of the field

Scaling LLMs faces dual bottlenecks: massive memory demands and computational inefficiency. Traditional frameworks impose rigid boundaries, localized memory caps capacity and trigger recomputations, while processor level kernel barriers prevent compute and network overlap. Overcoming this requires treating clusters and GPU processors as unified, asynchronous resource pools. Historically, optimization was siloed. Memory systems like vLLM coupled prefill and decoding, restricting prefix caching to local nodes and forcing expensive recomputations as context grew. Meanwhile, compilers relied on a rigid "kernel-per-operator" model where implicit barriers forced sequential execution, blocking cross-task pipelining and compute-communication overlap.

MOONCAKE resolves memory limits via a KVCache-centric disaggregated architecture. By pooling cluster-wide CPU, DRAM, SSD, and RDMA resources into a distributed cache, it dynamically migrates blocks across separated prefill and decode instances, trading storage for reduced computation. Conversely, MPK eliminates processor-level barriers by compiling multi-GPU inference into a single persistent mega-kernel. Using an SM-level graph, its in-kernel runtime asynchronously schedules tasks, enabling continuous software pipelining and concurrent compute-communication overlap. Recent systems extend these architectures: Cluster and chip-level scheduling are converging, with Nexus (2025) blending MOONCAKE's structural separation with MPK's fine-grained control. The mega-kernel has transitioned into a generalized standard, reflecting a paradigm shift toward treating memory, network, and compute as deeply intertwined pipelines.

2 Lecture papers

2.1 MPK: A Compiler and Runtime for Mega-Kernelizing Tensor Programs

What is the problem of this paper? Traditional "kernel-per-operator" execution in LLM serving creates rigid synchronization boundaries and idle hardware bubbles across Streaming Multiprocessors (SMs). Breaking tasks into finer granularities could eliminate these bubbles, but managing these dependencies via standard CPU-GPU synchronization introduces catastrophic scheduling overheads.

What is the solution of this paper? Mirage Persistent Kernel (MPK) is a compiler and runtime that transforms multi-GPU workloads into a single on-device "mega-kernel," bypassing CPU synchronization via three pillars:

SM-Level tGraph: Models computation at SM granularity. It treats inter-GPU communication as direct device-to-device memory accesses, bypassing the host CPU entirely. This allows the system to seamlessly overlap network communication with ongoing computation.

MPK Compiler: Fuses multiple tensor operations into an optimized tGraph. This fusion is necessary to eliminate redundant global memory round-trips and expose the fine-grained hardware parallelism that traditional operator boundaries block.

In-Kernel Parallel Runtime: Partitions SMs into compute *workers* and dispatch *schedulers*.

How is the solution evaluated? MPK is evaluated on auto-regressive LLM inference across single and multi-GPU topologies against state-of-the-art baselines. Key Results: MPK achieves a 1.0× to 1.7× latency reduction. Keeping execution entirely on-device eliminates kernel launch overheads and improves GPU utilization.

Overall, what are 1-2 shortcomings of this paper? On-Device Resource Contention: Permanently partitioning SMs, shared memory, and registers for the on-device scheduler restricts raw computation resources. This limits peak throughput for highly arithmetically intensive workloads where kernel launch overheads are negligible. Loss of Hardware Observability: Fusing workloads breaks standard profilers, making debugging exceptionally difficult since MPK never returns control to the CPU.

2.2 Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot

What is the problem of this paper? Large language model (LLM) serving providers must maximize throughput under strict latency service-level objectives (SLO), mainly time-to-first-token (TTFT) and time-between-tokens (TBT). LLM inference is split into a prefill phase, which processes the initial prompt and generates the first token, and a decode phase, which computes the rest of the output tokens. Prefill is compute-bound and decoding is memory-bound, making SLOs difficult to meet due to long-context inputs.

What is the solution of this paper? Mooncake’s key design philosophy is to “trade more storage for less computation” by using a large distributed KV cache. Mooncake disaggregates the prefill and decode nodes, underused CPU, DRAM, SSD, and RDMA resources into a global KV cache store. Every prefill and decode node contributes its DRAM to the pool of distributed KV caching, and KV blocks are keyed with their hash and prefix so any node can easily locate and fetch them over RDMA. A global scheduler routes inference requests to the prefill node with the lowest estimated TTFT, based on the longest KV block prefix matching and load intensity. If the chosen prefill node holds a shorter prefix than the best holder, then it retrieves the missing KV cache blocks over RDMA. Uncached KV is computed and written back to DRAM, and the final full KV cache is streamed to a decode node, overlapping with remaining prefill. The decode node, chosen based on load, then joins continuous batching as described in Orca and vLLM.

How is the solution evaluated? Experiments run on 16 nodes of 8xA800 GPUs with 4x200 Gbps NICs using a dummy LLaMA3-70B, compared against vLLM. The primary metric is effective request capacity, prefill GPU time, cache hit rate, transfer latency, bandwidth sensitivity, and P/D ratio. Mooncake reached 59-498% higher effective capacity than vLLM, 36-64% less prefill GPU time, and transferred 2.4-4.6x faster than TCP. The global cache pool hit rate increased up to 2.36x and prefill time decreased up to 48% compared to local caching.

Overall, what are 1-2 shortcomings of this paper? The only baseline Mooncake compared to was vLLM, making it unclear whether the increases in performance were due to disaggregation from the global KV store or from cache-aware request scheduling. In addition, they rely on a

high bandwidth network interconnect, making Mooncake less widely applicable for heterogeneous or shared clusters.

3 Lecture discussions

Question/Discussion 1: Why couldn't execution frameworks overlap operations previously? They were restricted by rigid kernel boundaries that forced operations to execute sequentially.

Question/Discussion 2: Can operations be reordered within a single kernel? Yes, operations can be scheduled internally in any order based on data dependencies and communication needs.

Question/Discussion 3: Does this architecture assume requests might hit different machines in a cluster? Yes, requests can be routed to different machines across the distributed cluster.

Question/Discussion 4: How does the system know where specific cached data exists? A central "Conductor" maintains a global list of available cache blocks.

Question/Discussion 5: Could MOONCAKE benefit from a cache eviction policy better than Least Recently Used? Cache eviction policy is found to be good for MOONCAKE, since agentic AI workloads have a cache hit rate of 90%+.

Question/Discussion 6: Are there workloads where MOONCAKE performs poorly, and does overhead ever outweigh recomputation? It struggles with highly varied topics where prefix matches drop to zero, shifting the bottleneck to the Conductor. However, even with 0% cache hits, decoding speeds still improve over baselines.

Question/Discussion 7: How does MOONCAKE handle less than ideal network bandwidth? If slow networks cause it to miss Service Level Objectives, MOONCAKE rejects requests.

Question/Discussion 8: In what scenarios will MPK fail or underperform? It struggles if models don't benefit from pipelining, or in highly dynamic workloads like MoE where constantly reallocating SMs introduces heavy overhead.

4 Proposal for follow-up work

MPK and Mooncake are similarly aimed at reducing time spent on computation, but do so by attacking distinct parts of the inference stack, where the Mirage Persistent Kernel is centralized to allow for *optimizations in computation* while Mooncake distributes the KV cache to *avoid recomputation*. Because both these systems exist with proven benefits to inference throughput, it follows logically that Mooncake's hardwired tradeoffs could be exposed to the same scheduler that handles MPK's SM task graph, allowing a runtime decision on "reuse" vs. "recompute".

This reasoning-based inference runtime model would make reasoning state a schedulable object filled with relevant variables like shared prefixes, branches, and cache dependencies, so that whenever the required amount of KV state changes, the reasoning model would be able to make a predictive probabilistic decision on what to do with a KV block based on how likely it is to be

needed, and how soon. Integrating this process into MPK’s scheduler would place KV-cache operations and the GPU computation relying on it within a single dependency-aware system.

For example, at runtime, the reasoning structure could decide not to persist branches likely to be abandoned, or prioritize caching and prefetching a common dependency, while demoting eventually useful states to SSD to save DRAM. Making the scheduler aware of cache dependencies also allows for cache operations to overlap with GPU computations. This is not a naive combination of Mooncake and MPK, but rather augments the capabilities of each to allow for an entirely new way of dynamically allocating inference-time resources through cache-aware scheduled operations: a level of storage optimization which neither paper explored.

5 A reflection on AI usage

AI is very useful for some tasks, in particular information retrieval and condensation. We observed that the quality of the final summary improved when there was access to LLM tools, and the time required decreased considerably.

However, the technology is not perfect, some of the issues we found were the following: Prompts are not always strictly followed so it is very important to double check the results. AI models can also get “stuck”, at some points we asked the agent to make the summary shorter without removing critical information, but it was unable to do so. We also noticed that it is easy to become complacent in using this technology.

Finally, we found some strategies that helped us obtain better responses with the prompts: challenge AI (we asked it to imagine it had a mentor critiquing its work, which helped refine results), iterating and asking clarifying questions and providing restrictions and examples on how it should execute its tasks.